

TP5 filtering

We will implement several ways to test the filter on arduino.

The filter is a moving average between 4 successive samples.

Exercise 1: filter analysis

1. What are the coefficients a and b of this filter ?
2. Plot the frequency response of this filter.
3. What kind of filter is it ?
4. What is its cut-off frequency at -3 dB?

Exercise 2: the gold model

Before implementing the filter on the microcontroller, it is a good idea to simulate the filter on the PC.

This gold model will be used as a reference when you will test your implementation on the Arduino

1. How many bits are available on the Arduino ADC ?
2. What is the output dynamic range of the Arduino ADC ?
3. Are these numbers signed or unsigned ?
4. The input data, x, of the gold model will be unsigned integers, The filter computation, y, will be float but the filter output, yq, will be unsigned integers.

Write a python code which describes this behaviour.

Test it on this set of arduino ADC like data :

`x=np.array((512,369,237,127,48,6,4,43,119,226,356,498,642,775,887,970,1016,1021,987,914))`

5. implement the same thing on the arduino and print your result on the Serial monitor. You should get the same output as the gold model.
You will use that function which simulates the ADC output :

```
unsigned int ADCread(long n){ //n is the time index
  unsigned int data[20] = {512,369,237,127,48,6,4,43,119,226,356,498,642,775,887,970,1016,1021,987,914};
  if (n<20) {
    return data[n]; //simulate analogRead(A0)
  }
  else {
    return 0;
  }
}
```

Exercise 3: Better filter simulation

the testing with true signals (sine wave or Gaussian white noise) is more reliable.

1. What is the input dynamic range of the Arduino ADC ?
2. Write a python function that emulate the arduino ADC. The input data is a float sample simulating the analog data, the output is a integer representing the number provides by the ADC.
3. Generate a float sinewave or a white Gaussian noise, apply your ADC function and analyze the resulting signal (time and frequency)
4. apply the filter with float coefficients and round the result over 16 bits :
5. analyse the resulting signal (time and frequency)

Exercise 4: Improve the output snr

Find a more optimal way to quantize the result over 16 bits (optimal = better SNR).

1. implement your approach in the gold model to test your approach. Compare with and without this optimization.
2. implement it in the Arduino
3. you can also change the input quantization. Explain what happens.

Exercise 5: Better arduino implementation testing

The workflow is:

1. **with Python, generate a true signal and store it in a text file**
2. **With a Serial App (Real Term), send this file into the arduino**
 - a) in the tab "Port", configure the serial communication and click on "open" button
 - b) in the tab "Capture", define a text file and click on "Start overwrite" button
 - c) in the tab "Send", define a text file (test.txt for example) and click on "Send File" button (choose a baudrate of 115200, and put 5 ms between each sending in RealTerm)
 - d) in the tab "Capture", click on "Stop Capture" button
 - e) in the tab "Port", click on "open" button
3. in the arduino read the data through the serial library (with also 115200 bauds)
4. Filter the data
5. Send back the result to the Serial link
6. with the Serial app, store the results
7. in python, the result file and compare it with the gold model

Exercise 6: a more complex filter

Now, the filter is a moving average between 100 successive samples.

1. What are the coefficients a and b of this filter ?
2. Plot the frequency response of this filter.
3. What kind of filter is it ?
4. What is its cut-off frequency at -3 dB?
5. Implement this filter on the Arduino